

Panduan dosen Lab 07 - BaaS, Auth, RLS, dan pgvector

COMP6991031 sesi 07. Kasus: Alice dan Bob menyimpan catatan dalam satu database, tetapi satu staf tidak boleh membaca atau membuat catatan atas nama staf lain. Panduan ini mendampingi [modul mahasiswa dengan kunci A-F](#). RPS sesi 07 mencakup BaaS/Supabase, schema SQL, Auth, RLS, Storage, dan pengantar pgvector. Praktik inti di sini berjalan pada PostgreSQL lokal; proyek Supabase adalah perluasan bila akun tersedia.

Persiapan dosen sebelum kelas

1. Pastikan Docker Engine aktif dan port `127.0.0.1:8087` kosong. Buka root repo `meet7CloudService` di PowerShell.
2. Salin `secrets/db_password.example.txt` menjadi `secrets/db_password.txt`; isi nilai latihan lokal. File hasil salinan dan `.env` diabaikan Git.
3. Jalankan `docker compose config --quiet` lalu `docker compose --profile debug up -d --wait`. Buka Chrome pada `<http://127.0.0.1:8087/>`. Jangan tampilkan password saat berbagi layar.
4. Jalankan `local-rls-demo.sql` dan `vector-local-demo.sql` dengan command di bawah. Checker `tests/challenge.ps1` harus mencapai 10 PASS.
5. Siapkan dua jendela: terminal untuk SET ROLE dan Chrome/Adminer untuk tabel owner. Kontras kedua tampilan adalah inti penjelasan RLS.

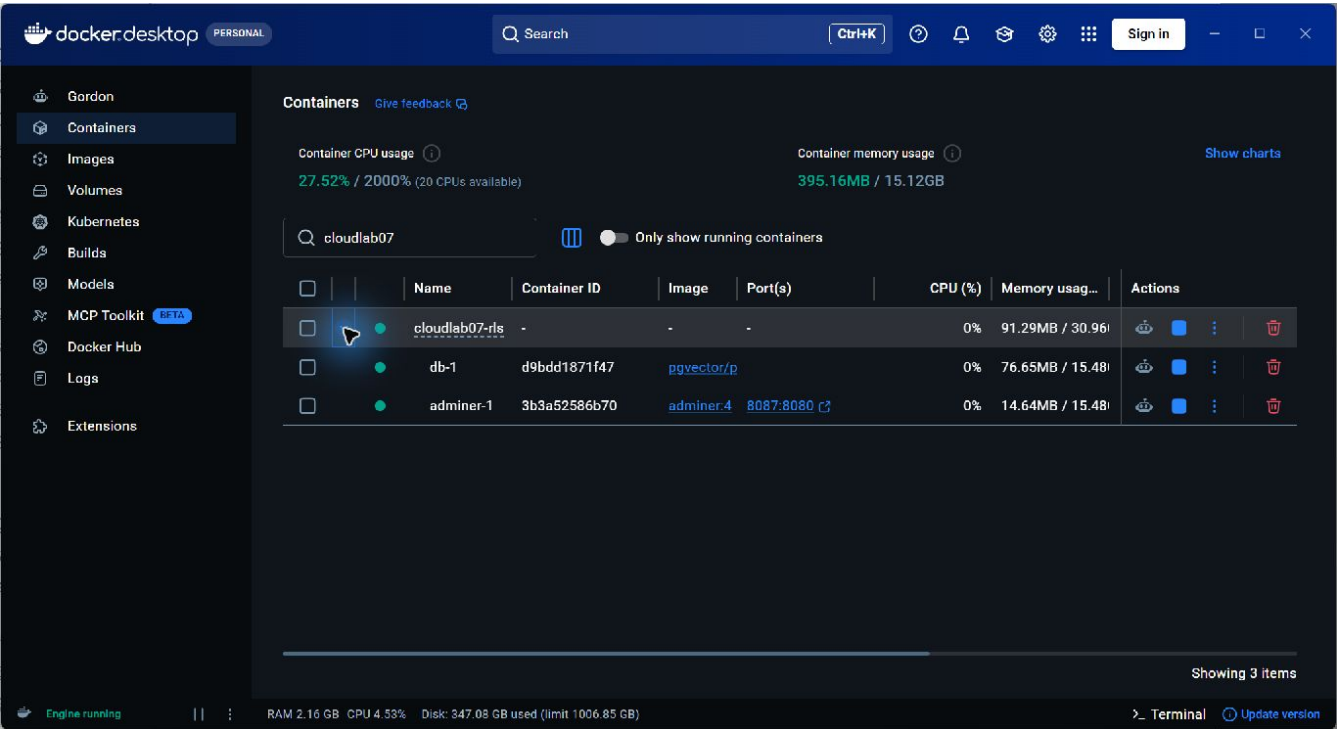
```
LAB 07 ? database dan Adminer sehat

$ docker compose --profile debug up -d --wait
db Healthy; adminer Healthy
$ docker compose ps
```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
cloudlab07-rls-adminer-1	127.0.0.1:8087->8080/tcp	adminer:4	"entrypoint.sh docke..."	adminer	3 minutes ago	Up 3 minutes
cloudlab07-rls-db-1 (healthy)	5432/tcp	pgvector/pgvector:pg16	"docker-entrypoint.s..."	db	3 minutes ago	Up 3 minutes

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: docker compose --profile debug up -d --wait lalu docker compose ps. Fungsi: memastikan db siap sebelum demo SQL. Cara kerja: Compose menunggu healthcheck PostgreSQL, membuat jaringan dan volume, dan hanya membuka Adminer di host 8087. Baca hasil: db Healthy, Adminer Up, port 5432 tidak dipublish.



Langkah: buka Containers di Docker Desktop dan perluas cloudlab07-r1s. Fungsi: memperlihatkan komponen lab kepada kelas. Cara kerja: Compose mengelompokkan db-1 dan adminer-1; titik hijau berarti container berjalan. Baca hasil: database memakai image pgvector/pgvector:pg16, Adminer memakai adminer:4 dengan port host 8087. Jelaskan bahwa status Healthy yang lebih kuat diperiksa melalui docker compose ps.

Alokasi 100 menit yang disarankan

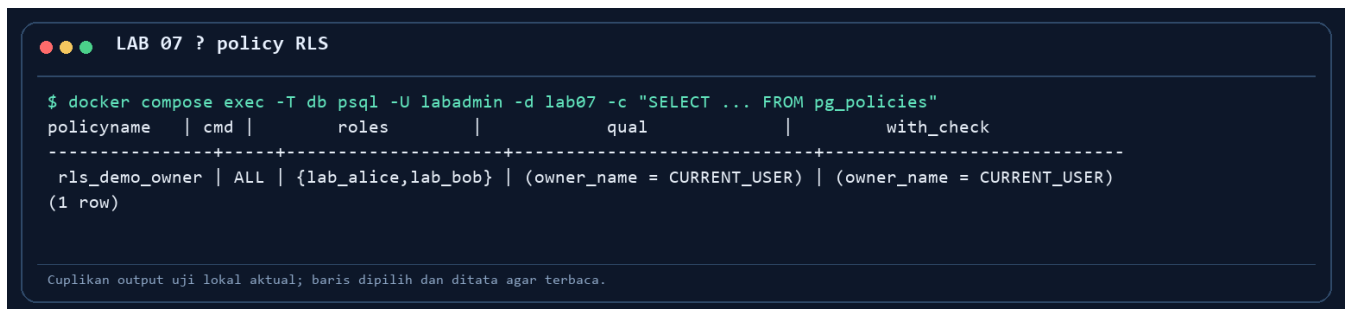
Menit	Aktivitas dosen	Bukti/checkpoint mahasiswa
0-10	Ceritakan kasus kebocoran catatan antarstaf; bedakan Auth dan otorisasi.	Mahasiswa menjelaskan mengapa filter frontend saja tidak cukup.
10-25	Start Compose, bahas secret file, healthcheck, dan Adminer.	docker compose ps menunjukkan DB sehat.
25-45	Jalankan SQL, baca GRANT, USING, WITH CHECK.	Policy terpasang.
45-60	Demo SELECT Alice/Bob dan INSERT lintas pemilik yang ditolak.	Dua baris terisolasi; error RLS terkontrol.
60-75	Buka Adminer sebagai owner, bandingkan dengan SET ROLE.	Mahasiswa memahami owner bypass.
75-85	Jalankan pgvector mainan dan urutan jarak.	Docker paling dekat dengan query.
85-100	Jalankan checker, diskusikan kunci A-F, laporan/Git/cleanup.	10 PASS, 0 FAIL dan bukti pribadi tersimpan.

Demo 1 - setup dan policy

PowerShell dari root repo:

```
Copy-Item .\secrets\db_password.example.txt .\secrets\db_password.txt
docker compose --profile debug up -d --wait
Get-Content -Raw .\local-rls-demo.sql | docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin \
-d lab07
docker compose exec -T db psql -U labadmin -d lab07 -c "SELECT polycyname, cmd, roles, qual,
\with_check FROM pg_policies WHERE tablename='rls_demo';"
```

Jika file password sudah ada, jangan menyimpannya tanpa sengaja. Pada Bash, gunakan redirection < `local-rls-demo.sql`. -T diperlukan saat psql membaca pipe. Script membuat dua role dan tabel `rls_demo`, mengaktifkan RLS, memberi GRANT, memasang policy, lalu menyiapkan satu catatan per role. Demo dapat diulang; hanya tabel ini yang di-TRUNCATE.



```
$ docker compose exec -T db psql -U labadmin -d lab07 -c "SELECT ... FROM pg_policies"
polycyname | cmd | roles | qual | with_check
-----+-----+-----+-----+-----
rls_demo_owner | ALL | {lab_alice,lab_bob} | (owner_name = CURRENT_USER) | (owner_name = CURRENT_USER)
(1 row)
```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: baca `pg_policies` setelah script. Fungsi: memeriksa aturan database, bukan hanya klaim dalam SQL file. Cara kerja: USING menyaring baris lama dan WITH CHECK memvalidasi baris baru/hasil UPDATE. Baca hasil: `owner_name = CURRENT_USER` untuk kedua role.

Pertanyaan cepat: “Apakah GRANT SELECT otomatis membuat Alice hanya melihat barisnya?” Jawaban: tidak; GRANT bekerja pada tabel, RLS pada baris. Jika mahasiswa melihat 0 baris, cek SET ROLE, nilai `owner_name`, serta policy. Jika melihat 2 baris sebagai `labadmin`, itu owner bypass; lanjutkan dengan SET ROLE.

Demo 2 - bukti isolasi dan penolakan

```
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 -c "SET ROLE lab_alice; SELECT
\current_user AS viewer, owner_name, note FROM rls_demo; RESET ROLE; SET ROLE lab_bob; SELECT
\current_user AS viewer, owner_name, note FROM rls_demo;"
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 -c "SET ROLE lab_bob; INSERT
\INTO rls_demo (owner_name,note) VALUES ('lab_alice','Percobaan silang');"
```

Command kedua **harus gagal** dengan pesan `new row violates row-level security policy`. Jangan mengubah policy agar tes merah itu “lulus”; kesalahan tersebut adalah bukti keamanan yang diminta. Bedakan kegagalan terencana ini dari masalah instalasi.

LAB 07 ? dua identitas, hasil terpisah

```
$ docker compose exec -T db psql ... -c "SET ROLE lab_alice; SELECT ...; SET ROLE lab_bob; SELECT ..."
SET
 viewer | owner_name | note
-----+-----+-----
 lab_alice | lab_alice | Catatan Alice
(1 row)

RESET
SET
 viewer | owner_name | note
-----+-----+-----
 lab_bob | lab_bob | Catatan Bob
(1 row)
```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: SET ROLE Alice dan Bob sebelum SELECT. Fungsi: menunjukkan otorisasi di database. Cara kerja: policy memfilter hasil untuk current_user. Baca hasil: masing-masing tepat satu catatan miliknya.

LAB 07 ? INSERT lintas pemilik ditolak

```
$ docker compose exec -T db psql ... -c "SET ROLE lab_bob; INSERT ... owner_name=lab_alice"
SET
ERROR: new row violates row-level security policy for table "rls_demo"
[exit code 1]
```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: SET ROLE lab_bob lalu INSERT dengan owner_name='lab_alice'. Fungsi: menguji WITH CHECK. Cara kerja: PostgreSQL menolak baris yang tidak memenuhi policy. Baca hasil: error RLS dan exit code bukan nol; jumlah baris tetap dua.

Demo 3 - Adminer dan pgvector

Di Adminer, pilih PostgreSQL/server db/user labadmin/database lab07, lalu tabel rls_demo. Owner tabel melihat kedua baris. Mintalah mahasiswa membandingkan identitas login Adminer dengan viewer pada terminal. Jangan gunakan UI owner sebagai satu-satunya bukti RLS.

Language: English PostgreSQL > db > lab07 > public > Select: rls_demo labadmin Logout

Adminer 4.17.1 6.1.1

DB: lab07 Schema: public

SQL command Import
Export Create table

select lab_embeddings
select rls_demo

Select: rls_demo

Select data Show structure Alter table New item

Select Search Sort Limit 50 Text length 100 Action Select

SELECT * FROM "rls_demo" LIMIT 50 (0.001 s) Edit

	id	owner_name	note
☐ edit	a15b1619-5f7e-456b-a9b7-d520b86df32	lab_alice	Catatan Alice
☐ edit	170b0c73-378c-4e96-8d90-146499de194d	lab_bob	Catatan Bob

Whole result ☐ 2 rows Modify Selected (0) Export (2)

Import

Langkah: buka 127.0.0.1:8087 dan klik select rls_demo. Fungsi: memperlihatkan data nyata di web. Cara kerja: Adminer mengakses PostgreSQL melalui jaringan Compose sebagai owner. Baca hasil: dua baris terlihat; hasil ini konsisten dengan query role yang masing-masing satu baris.

```
Get-Content -Raw .\vector-local-demo.sql | docker compose exec -T db psql -v ON_ERROR_STOP=1 -U \labadmin -d lab07
```

File ini mengaktifkan pgvector, membuat tiga vektor mainan, lalu mengurutkan jarak ke [1,0,0]. Kaitkan operator <-> dengan gagasan pencarian top-K yang akan muncul lagi pada Lab 11. Jangan menyebut tiga angka buatan itu sebagai embedding model.

LAB 07 ? urutan jarak vektor

```
$ docker compose exec -T db psql ... -c "SELECT judul, jarak ... ORDER BY embedding <-> query"
judul | jarak
-----+-----
Docker | 0.000
Container | 0.141
Database | 1.414
(3 rows)
```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: jalankan script vektor. Fungsi: melihat nearest neighbor sederhana. Cara kerja: <-> menghitung jarak Euclidean dan ORDER BY mengurutkan. Baca hasil: Docker 0, Container sekitar 0,141, Database sekitar 1,414.

Language: English PostgreSQL » db » lab07 » public » Select: lab_embeddings labadmin Logout

Adminer 4.17.1 6.1.1

DB: lab07 Schema: public

[SQL command](#) [Import](#)
[Export](#) [Create table](#)

select lab_embeddings
select rls_demo

Select: lab_embeddings

Select data [Show structure](#) [Alter table](#) [New item](#)

SELECT * FROM "lab_embeddings" LIMIT 50 (0,001 s) Edit

☐	Modify	id	judul	embedding
☐	edit	1	Docker	[1,0,0]
☐	edit	2	Container	[0.9,0.1,0]
☐	edit	3	Database	[0,0,1]

☐ 3 rows

[Import](#)

Langkah: klik `select lab_embeddings`. Fungsi: mengaitkan hasil query dengan tabel. Cara kerja: Adminer membaca nilai vector(3) sebagai teks. Baca hasil: tiga baris ada; kemiripan tidak dibuktikan oleh urutan default tabel.

Kunci challenge A-F untuk diskusi

Bagian	Kunci
A	<code>up -d --wait</code> menunggu healthcheck; <code>ps</code> menunjukkan database Healthy.
B	GRANT membuka operasi tabel, RLS menyaring baris; keduanya perlu.
C	labadmin adalah owner dan dapat melihat dua baris; <code>SET ROLE lab_alice/bob</code> masing-masing melihat satu.
D	<code>USING</code> mengatur baris lama, <code>WITH CHECK</code> baris baru; <code>INSERT</code> silang ditolak.
E	Docker paling dekat dengan <code>[1,0,0]</code> . Vektor tiga dimensi ini contoh numerik, bukan model ML.
F	Lokal memakai <code>current_user</code> ; Supabase memakai <code>JWT + auth.uid()</code> . Secret/service role key tidak boleh di browser.

Verifikasi: & `.\tests\challenge.ps1` di PowerShell atau `bash tests/challenge.sh` di Git Bash/Linux. Dua skrip diuji di laptop dosen dan menghasilkan **10 PASS, 0 FAIL**.



```

$ powershell.exe -NoProfile -Command "& .\tests\challenge.ps1"
PASS PostgreSQL 16+
PASS dua role demo
PASS RLS aktif
PASS Alice hanya lihat miliknya
PASS Bob hanya lihat miliknya
PASS Bob tidak lihat Alice
PASS jumlah baris demo tetap dua
PASS extension pgvector aktif
PASS tetangga terdekat Docker
PASS INSERT lintas pemilik ditolak
Lab 07: 10 PASS, 0 FAIL

```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: jalankan checker. Fungsi: mengulang bukti role, policy, vector, dan penolakan. Cara kerja: skrip memanggil psql dan menganggap error INSERT silang sebagai PASS. Baca hasil: 10 PASS, 0 FAIL.

Supabase, penilaian, dan Git

Pada jalur cloud, `schema.sql` harus dijalankan di **Supabase SQL Editor** karena memakai `auth.users` dan `auth.uid()`; file itu tidak sesuai untuk database Compose lokal. Minta mahasiswa memakai dua akun latihan, publishable key, dan `.env` lokal. Storage bucket private saja belum memberi akses aplikasi tanpa policy. Bila akun/kuota tidak tersedia, nilai capaian inti dari jalur lokal dan diskusi desain Supabase.

Laporan yang baik berisi fungsi tiap command, SELECT terpisah, error INSERT yang diharapkan, screenshot Adminer dengan penjelasan owner, urutan vektor, dan refleksi bagaimana database mencegah kebocoran bila frontend salah. Mahasiswa membuat repo pribadi dari template, memeriksa `git status` dan `git diff --cached --name-only`, lalu push laporan serta screenshot **milik sendiri**. Pastikan `.env` dan `secrets/db_password.txt` tidak staged.

Sesudah kelas, jalankan `docker compose --profile debug down`. Volume tetap ada; jangan gunakan `-v` bila data hendak dipakai lagi.

Diagnosis cepat

Gejala	Arahkan mahasiswa
Cannot connect to Docker daemon	Buka Docker Desktop hingga Engine running; ulangi <code>docker version</code> .
Password secret tidak ada	Salin contoh ke <code>secrets/db_password.txt</code> , jangan commit.
Port 8087 dipakai	Periksa <code>docker ps</code> dan pemilik port; jangan hentikan service yang bukan lab.
<code>auth.users</code> tidak ada	Mereka menjalankan SQL Supabase pada PostgreSQL lokal; gunakan <code>local-rls-demo.sql</code> .
Adminer dua baris	Jelaskan owner bypass, ulangi SELECT memakai <code>SET ROLE</code> .
Checker vector gagal	Jalankan <code>vector-local-demo.sql</code> ; cek extension <code>vector</code> .

Rujukan: [Supabase RLS](#), [Supabase vector columns](#), [pgvector](#).